Module 07 Lab - Better Model Evaluation**Objective:** To move beyond simple accuracy and learn how to use more sophisticated and reliable evaluation techniques, including the confusion matrix, precision, recall, and cross-validation.**In this lab, you will write the code to generate and interpret these advanced evaluation metrics.**

Part 1: Setup and Model TrainingLet's first train a model so we have something to evaluate. We will use the Titanic dataset again to predict survival.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

# Load and prepare data
df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv')

df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})
df['Age'] = df['Age'].fillna(df['Age'].median())

features = ['Age', 'Pclass', 'Sex', 'Fare']
target = 'Survived'

X = df[features]
y = df[target]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train a model
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

Part 2: The Confusion Matrix**Concept:** A confusion matrix gives you a more detailed breakdown of a model's performance than accuracy alone. It's a table that shows you where your model got things right and where it got them wrong.It has four quadrants:* **True Positives (TP):** Correctly predicted positive (e.g., predicted survival, and they did survive).* **True Negatives (TN):** Correctly predicted negative (e.g., predicted did not survive, and they did not).* **False Positives (FP):** Incorrectly predicted positive (e.g., predicted survival, but they did not). Also called a "Type I Error".* **False Negatives (FN):** Incorrectly predicted negative (e.g., predicted did not survive, but they did). Also called a "Type II Error".

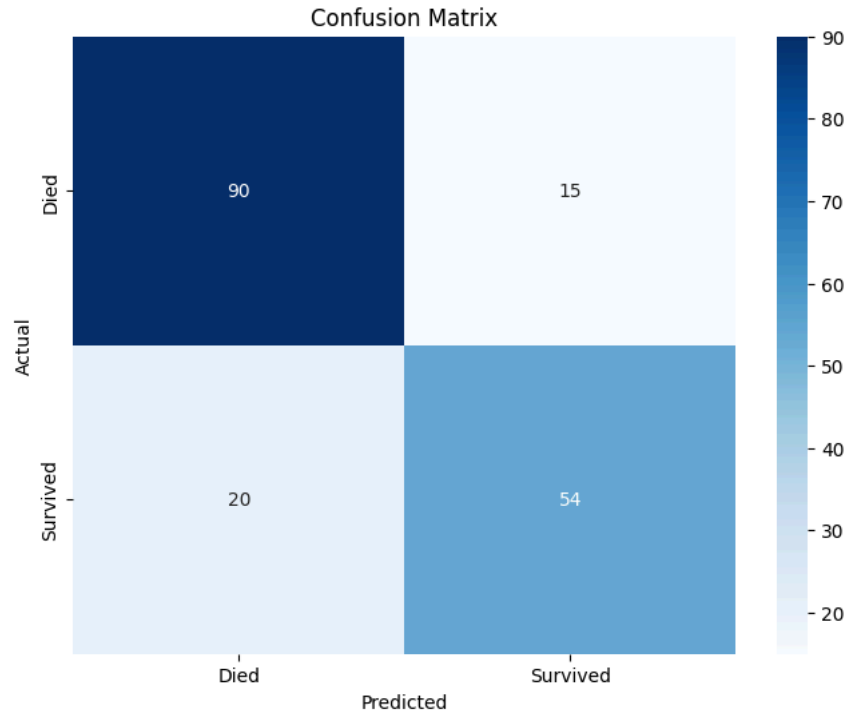
Task 1: Generate and Plot a Confusion Matrix**Your Task:** Use `confusion_matrix` from `sklearn.metrics` to calculate the matrix and `seaborn.heatmap` to visualize it.

```
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# 1. Calculate the confusion matrix
cm = confusion_matrix(y_test, y_pred)

# 2. Create a heatmap to visualize it
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues'
```

```
            xticklabels=['Died', 'Survived'],
            yticklabels=['Died', 'Survived'])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()
```



Part 3: Precision, Recall, and F1-Score**Concept:** From the confusion matrix, we can calculate more nuanced metrics:* **Precision:** Of all the times the model predicted **positive**, how often was it correct? * Formula: `TP / (TP + FP)` * *Use Case:* When the cost of a **False Positive** is high. (e.g., a spam filter; you don't want to incorrectly mark an important email as spam).* **Recall (Sensitivity):** Of all the actual **positives**, how many did the model correctly identify? * Formula: `TP / (TP + FN)` * *Use Case:* When the cost of a **False Negative** is high. (e.g., a medical test for a serious disease; you don't want to miss a real case).* **F1-Score:** The harmonic mean of Precision and Recall. It provides a single score that balances both.

Task 2: Generate a Classification Report**Your Task:** Use `classification_report` from `sklearn.metrics` to get a summary of these metrics for each class.

```
from sklearn.metrics import classification_report

# 1. Generate and print the classification report
report = classification_report(y_test, y_pred, target_names=['Died', 'Survived'])
print(report)
```

```
              precision    recall  f1-score   support

        Died       0.82      0.86      0.84       105
    Survived       0.78      0.73      0.76        74

    accuracy                           0.80       179
   macro avg       0.80      0.79      0.80       179
weighted avg       0.80      0.80      0.80       179
```

Part 4: Cross-Validation**Concept:** A single train-test split can be lucky or unlucky. What if, by chance, all the "easy" examples ended up in our test set? Our accuracy score would be misleadingly high.**Cross-Validation (CV)** solves this. It splits the data into multiple "folds" (e.g., 5 or 10). It then trains and evaluates the model multiple times, using a different fold as the test set each time. The final score is the average of the scores from all folds.This gives a much more robust and reliable estimate of the model's true performance.

Task 3: Perform Cross-Validation**Your Task:** Use `cross_val_score` from `sklearn.model_selection` to perform 5-fold cross-validation on your model.

```
from sklearn.model_selection import cross_val_score

# 1. Perform 5-fold cross-validation
cv_scores = cross_val_score(model, X, y, cv=5)

# 2. Print the scores for each fold and the average score
print(f"Scores for each fold: {cv_scores}")
print(f"Average CV Score: {cv_scores.mean():.2%}")
print(f"Standard Deviation of CV Scores: {cv_scores.std():.4f}")
```

```
Scores for each fold: [0.7877095  0.78089888 0.78651685 0.7752809  0.80337079]
Average CV Score: 78.68%
Standard Deviation of CV Scores: 0.0094
```

1. Real-world scenario where I care more about Precision than Recall:Mathematical Definition:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

This measures the proportion of positive predictions that are actually correct, directly penalizing Type I errors (False Positives).Scenario & Justification:I prioritize precision in automated fraud detection when automatically freezing accounts or in email spam filtering. If my model predicts fraud and automatically locks a legitimate user's bank account, or flags a critical client email as spam (a False Positive), it causes immediate disruption, loss of trust, and operational overhead. Because a Type I error carries a significantly higher penalty than letting a rare false negative slip through for secondary review, I tune my classification threshold to minimize False Positives and maximize Precision.2. Real-world scenario where I care more about Recall than Precision:Mathematical Definition:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

This measures the proportion of actual positive cases my model correctly identifies, directly penalizing Type II errors (False Negatives).Scenario & Justification:I prioritize recall in early cancer screening (e.g., detecting malignant tumors from CT scans). If my model produces a False Negative, it misdiagnoses an active malignancy as benign, delaying critical treatment and potentially resulting in patient mortality. Conversely, a False Positive merely triggers non-fatal follow-up testing (like a biopsy). Because the asymmetric cost of a Type II error is catastrophic, I lower the decision threshold to capture every true positive and drive Recall as close to 1.0 as possible.3. Why a cross-validation score is more trustworthy than a single train-test split:Variance Reduction: A single train-test split produces a high-variance performance estimate $\hat{\theta}$ conditioned on a single random partitioning seed. In k-fold cross-validation, I average performance across k non-overlapping holdout folds:

$$\hat{\theta}_{\text{CV}} = \frac{1}{k} \sum_{i=1}^{k} \hat{\theta}_i$$

By aggregating estimates across distinct subsets, the variance of my generalization error estimate is substantially lower ($\text{Var}(\bar{X}) = \frac{\sigma^2}{k}$).Mitigating Sampling Bias: Cross-validation prevents my metrics from being artificially inflated or penalized by an uncharacteristically "easy" or "noisy" test partition, ensuring every single observation is evaluated out-of-sample exactly once.Quantifiable Dispersion (σ): A single split provides only a point estimate. Cross-validation allows me to compute the standard deviation (σ_{CV}) across folds, providing empirical proof of whether my model is stable or sensitive to perturbations in the training data.